Laboratornaya_Rabota_... PDF

Tranzaktsionnaya_Pamy... PDF

Нужно сделать домашнее задание. Выполни его, пожалуйста, а я скопирую историю чата и отправлю в агента, чтобы он сделал. Как я понимаю, нам необходимо создать приложение на MAUI. Действуй!

claude-sonnet-5-high

Готовое решение — MAUI-приложение «STM & CAS Visual Demo»

Собрал полноценный проект на .NET MAUI, который реализует **оба примера из задания**:

- **Пример 1** — симуляция STM через CAS (`Interlocked.CompareExchange`), как в методичке, с визуализацией 100→N потоков, пополняющих один счёт.
- **Пример 2** — «настоящий» учебный STM-движок на чистом C# (т.к. `Nito.Transactions` / `MSR STM.NET` недоступны) по принципу **MVCC** (версионирование + commit-time validation), визуализирующий параллельный перевод денег между двумя счетами с сохранением инварианта суммы.

Всё обёрнуто в MVVM, с анимированной графикой на `GraphicsView` (потоки = кружки вокруг счёта, цвет = состояние: чтение/вычисление/конфликт-retry/коммит), логом событий и настройками (кол-во потоков, суммы).

---

1. Создание проекта

{} Bash

```bash
dotnet new maui -n StmVisualDemo
cd StmVisualDemo
```

Если имя проекта будет другим — поменяйте `namespace StmVisualDemo` и `xmlns:vm="clr-namespace:StmVisualDemo.ViewModels"` везде ниже.

2. Структура файлов

{} text

```text
StmVisualDemo/
 ├─ Models/
 │   ├─ CasAccount.cs
 │   ├─ TVar.cs
 │   └─ StmEngine.cs
 ├─ ViewModels/
 │   └─ MainViewModel.cs
 ├─ Views/
 │   └─ StmDrawable.cs
 ├─ MainPage.xaml
 └─ MainPage.xaml.cs
```

---

3. Models/CasAccount.cs

(реализация «Пример функционала 1» из методички)

{} csharp

```csharp
namespace StmVisualDemo.Models;

/// <summary>
/// Пример функционала 1 из задания: симуляция STM через CAS-операции
/// (Interlocked.CompareExchange) без использования lock.
/// Оптимистичная блокировка: читаем -> считаем -> пытаемся зафиксировать,
/// при неудаче - откат вычислений и повтор (retry).
/// </summary>
public sealed class CasAccount
{
    private int _balance;

    public int Balance => Volatile.Read(ref _balance);

    public int Deposit(int amount, Action? onConflict = null)
    {
        int attempts = 0;
        while (true)
        {
            attempts++;

            int current = _balance;            // 1. Начало "транзакции"
            int updated = current + amount;     // 2. Вычисление нового значения локально

            Thread.SpinWait(Random.Shared.Next(50, 300)); // Имитация полезной работы

            // 3. Попытка commit. Если _balance не изменился - фиксируем.
            if (Interlocked.CompareExchange(ref _balance, updated, current) == current)
                return attempts;

            // Конфликт: другой поток успел изменить баланс. Откат и retry.
            onConflict?.Invoke();
        }
    }
}
```

4. Models/TVar.cs

{} csharp

```csharp
namespace StmVisualDemo.Models;

/// <summary>
/// Упрощённая "транзакционная переменная" - аналог TVar (Haskell STM)
/// или Ref (Clojure/ScalaSTM), реализованная на чистом C#.
/// Подход MVCC: значение хранится вместе с номером версии,
/// вся ячейка (значение+версия) заменяется атомарно одной ссылкой.
/// </summary>
public sealed class TVar<T>
{
    private sealed record Cell(T Value, long Version);

    private Cell _cell;

    public TVar(T initial) => _cell = new Cell(initial, 0);

    /// <summary>Снимок значения и версии ("начало транзакции").</summary>
    public (T Value, long Version) Read()
    {
        var c = Volatile.Read(ref _cell);
        return (c.Value, c.Version);
    }

    /// <summary>
    /// Принудительная установка значения с увеличением версии.
    /// Вызывается только внутри критической секции коммита в StmEngine.
    /// </summary>
    internal void ForceSet(T value, long version) =>
        Volatile.Write(ref _cell, new Cell(value, version));
}
```

5. Models/StmEngine.cs

(реализация «Пример функционала 2» — аналог `TxContext.ExecuteInTransaction`)

{} csharp

```csharp
namespace StmVisualDemo.Models;

public readonly struct TxResult
{
    public bool Success { get; }
    public int Attempts { get; }
    public string? ErrorMessage { get; }

    private TxResult(bool success, int attempts, string? error)
    {
        Success = success;
        Attempts = attempts;
        ErrorMessage = error;
    }

    public static TxResult Ok(int attempts) => new(true, attempts, null);
    public static TxResult Fail(int attempts, string error) => new(false, attempts, error);
}

/// <summary>
/// Пример функционала 2 из задания: объединение изменений нескольких
/// связанных транзакционных переменных (TVar) в одну атомарную транзакцию
/// с автоматическим откатом (retry) при конфликте — упрощённый аналог
/// Nito.Transactions / TxContext.ExecuteInTransaction.
///
/// Алгоритм оптимистичной транзакционной памяти:
///  1. Снимок (Value, Version) для каждой переменной — "начало транзакции".
///  2. Вычисление новых значений локально, вне критической секции.
///  3. Валидация + Commit под короткой общей блокировкой: если версии
///     не изменились с момента чтения — обе переменные атомарно
///     обновляются; иначе — конфликт, полный перезапуск транзакции.
/// </summary>
public static class StmEngine
{
    private static readonly object CommitLock = new();

    public static TxResult Transfer(TVar<int> from, TVar<int> to, int amount, Action? onConflict = null)
    {
        int attempts = 0;

        while (true)
        {
            attempts++;

            var (fromVal, fromVer) = from.Read();
            var (toVal, toVer) = to.Read();

            if (fromVal < amount)
                return TxResult.Fail(attempts, "Недостаточно средств");

            int newFrom = fromVal - amount;
            int newTo = toVal + amount;

            // Имитация "тяжёлой" работы внутри транзакции — именно в этом
            // окне другой поток может успеть закоммититься и вызвать конфликт
            Thread.SpinWait(Random.Shared.Next(50, 400));

            lock (CommitLock)
            {
                var (curFromVal, curFromVer) = from.Read();
                var (curToVal, curToVer) = to.Read();

                if (curFromVer == fromVer && curToVer == toVer)
                {
                    from.ForceSet(newFrom, fromVer + 1);
                    to.ForceSet(newTo, toVer + 1);
                    return TxResult.Ok(attempts);
                }
            }

            // Валидация не прошла — версия изменилась, кто-то успел
            // закоммитить конфликтующую транзакцию. Откат и retry.
            onConflict?.Invoke();
        }
    }
}
```

6. ViewModels/MainViewModel.cs

{} csharp

```csharp
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows.Input;
using Microsoft.Maui.Controls;
using StmVisualDemo.Models;

namespace StmVisualDemo.ViewModels;

public enum DemoType { SingleAccountCas, TransferStm }
public enum WorkerState { Idle, Reading, Computing, Committing, Conflict, Success }

/// <summary>Визуальное состояние одного "потока" для отрисовки на GraphicsView.</summary>
public class WorkerVisual
{
    public int Id { get; init; }
    public volatile WorkerState State = WorkerState.Idle;
    public int Attempts;
    public int TargetAccount; // 0 -> B→A, 1 -> A→B (для демо 2)
}

public class MainViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;
    private void OnPropertyChanged([CallerMemberName] string? name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    #region Настройки

    private double _workerCount = 16;
    public double WorkerCount
    {
        get => _workerCount;
        set
        {
            if (Math.Abs(_workerCount - value) < 0.5) return;
            _workerCount = value;
            OnPropertyChanged();
            if (!IsRunning) ResetWorkers();
        }
    }

    private double _depositAmount = 10;
    public double DepositAmount
    {
        get => _depositAmount;
        set { if (Math.Abs(_depositAmount - value) < 0.5) return; _depositAmount = value; OnPropertyChanged(); }
    }

    private double _transferAmount = 20;
    public double TransferAmount
    {
        get => _transferAmount;
        set { if (Math.Abs(_transferAmount - value) < 0.5) return; _transferAmount = value; OnPropertyChanged(); }
    }

    private DemoType _selectedDemo = DemoType.SingleAccountCas;
    public DemoType SelectedDemo
    {
        get => _selectedDemo;
        set
        {
            if (_selectedDemo == value) return;
            _selectedDemo = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(IsSingleDemo));
            OnPropertyChanged(nameof(IsTransferDemo));
            OnPropertyChanged(nameof(ConflictsText));
            ResetAll();
        }
    }

    public bool IsSingleDemo => SelectedDemo == DemoType.SingleAccountCas;
    public bool IsTransferDemo => SelectedDemo == DemoType.TransferStm;

    #endregion

    public WorkerVisual[] Workers { get; private set; } = Array.Empty<WorkerVisual>();

    private bool _isRunning;
    public bool IsRunning
    {
        get => _isRunning;
        private set
        {
            _isRunning = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(IsNotRunning));
            (StartCommand as Command)?.ChangeCanExecute();
            (ResetCommand as Command)?.ChangeCanExecute();
        }
    }
    public bool IsNotRunning => !_isRunning;

    #region Лог

    private readonly object _logLock = new();
    private readonly List<string> _logLines = new();

    private string _logText = "Готов к запуску демонстрации...";
    public string LogText
    {
        get => _logText;
        private set { _logText = value; OnPropertyChanged(); }
    }

    private void AppendLog(string line)
    {
        lock (_logLock)
        {
            _logLines.Add($"[{DateTime.Now:HH:mm:ss.fff}] {line}");
            if (_logLines.Count > 300) _logLines.RemoveAt(0);
        }
    }

    /// <summary>Вызывается из UI-таймера для обновления графики/лога/статистики.</summary>
    public void RefreshStats()
    {
        OnPropertyChanged(nameof(SingleAccountBalance));
        OnPropertyChanged(nameof(AccountABalance));
        OnPropertyChanged(nameof(AccountBBalance));
        OnPropertyChanged(nameof(ConflictsText));

        lock (_logLock)
        {
            var snapshot = string.Join(Environment.NewLine, Enumerable.Reverse(_logLines));
            if (snapshot != _logText) LogText = snapshot;
        }
    }

    #endregion

    #region Демо 1: CAS

    private CasAccount _casAccount = new();
    public int SingleAccountBalance => _casAccount.Balance;

    private int _totalConflictsDemo1;

    #endregion

    #region Демо 2: STM (MVCC)

    private TVar<int> _accountA = new(1000);
    private TVar<int> _accountB = new(500);
    public int AccountABalance => _accountA.Read().Value;
    public int AccountBBalance => _accountB.Read().Value;

    private int _totalConflictsDemo2;

    #endregion

    public string ConflictsText => IsSingleDemo
        ? $"Конфликтов CAS (retry): {_totalConflictsDemo1}"
        : $"Конфликтов STM (retry): {_totalConflictsDemo2}, сумма A+B: {AccountABalance + AccountBBalance}";

    public ICommand StartCommand { get; }
    public ICommand ResetCommand { get; }

    public MainViewModel()
    {
        StartCommand = new Command(async () => await RunSelectedDemoAsync(), () => IsNotRunning);
        ResetCommand = new Command(ResetAll, () => IsNotRunning);
        ResetWorkers();
    }

    private void ResetWorkers()
    {
        int n = Math.Max(2, (int)WorkerCount);
        Workers = Enumerable.Range(0, n).Select(i => new WorkerVisual { Id = i }).ToArray();
        OnPropertyChanged(nameof(Workers));
    }

    private void ResetAll()
    {
        _casAccount = new CasAccount();
        _accountA = new TVar<int>(1000);
        _accountB = new TVar<int>(500);
        _totalConflictsDemo1 = 0;
        _totalConflictsDemo2 = 0;
        ResetWorkers();
        lock (_logLock) { _logLines.Clear(); }
        LogText = "Готов к запуску демонстрации...";
        RefreshStats();
    }

    private async Task RunSelectedDemoAsync()
    {
        if (IsRunning) return;
        IsRunning = true;
        try
        {
            if (SelectedDemo == DemoType.SingleAccountCas)
                await RunDemo1Async();
            else
                await RunDemo2Async();
        }
        finally
        {
            IsRunning = false;
            RefreshStats();
        }
    }

    private async Task RunDemo1Async()
    {
        _casAccount = new CasAccount();
        _totalConflictsDemo1 = 0;
        ResetWorkers();
        int amount = (int)DepositAmount;
        int n = Workers.Length;

        AppendLog("=== ДЕМО 1: CAS / Interlocked.CompareExchange ===");
        AppendLog($"Потоков: {n}, депозит каждого: {amount}, ожидаемый итог: {n * amount}");

        var tasks = Workers.Select(w => Task.Run(() =>
        {
            w.State = WorkerState.Reading;
            Thread.Sleep(Random.Shared.Next(5, 60));
            w.State = WorkerState.Computing;

            int attempts = _casAccount.Deposit(amount, onConflict: () =>
            {
                w.State = WorkerState.Conflict;
                Interlocked.Increment(ref _totalConflictsDemo1);
                Thread.Sleep(Random.Shared.Next(5, 25));
                w.State = WorkerState.Computing;
            });

            w.Attempts = attempts;
            w.State = WorkerState.Success;
            AppendLog($"Поток {w.Id,2}: +{amount}, попыток CAS: {attempts}");
        })).ToArray();

        await Task.WhenAll(tasks);

        AppendLog($"=== ИТОГ: баланс = {_casAccount.Balance} (ожидалось {n * amount}), retry:
{_totalConflictsDemo1} ===");
    }

    private async Task RunDemo2Async()
    {
        _accountA = new TVar<int>(1000);
        _accountB = new TVar<int>(500);
        _totalConflictsDemo2 = 0;
        ResetWorkers();
        int amount = (int)TransferAmount;
        int n = Workers.Length;
        int sumBefore = _accountA.Read().Value + _accountB.Read().Value;

        AppendLog("=== ДЕМО 2: STM-перевод (MVCC: версии + commit-time validation) ===");
        AppendLog($"Потоков: {n}, сумма перевода: {amount}, A0={_accountA.Read().Value}, B0=
{_accountB.Read().Value}");

        var tasks = Workers.Select(w => Task.Run(() =>
        {
            bool aToB = w.Id % 2 == 0;
            w.TargetAccount = aToB ? 1 : 0;

            w.State = WorkerState.Reading;
            Thread.Sleep(Random.Shared.Next(5, 60));
            w.State = WorkerState.Computing;

            var from = aToB ? _accountA : _accountB;
            var to = aToB ? _accountB : _accountA;

            var result = StmEngine.Transfer(from, to, amount, onConflict: () =>
            {
                w.State = WorkerState.Conflict;
                Interlocked.Increment(ref _totalConflictsDemo2);
                w.State = WorkerState.Computing;
            });

            w.Attempts = result.Attempts;
            w.State = result.Success ? WorkerState.Success : WorkerState.Conflict;

            AppendLog(result.Success
                ? $"Поток {w.Id,2}: перевод {(aToB ? "A→B" : "B→A")} {amount}, попыток: {result.Attempts}"
                : $"Поток {w.Id,2}: ОТКАЗ — {result.ErrorMessage}");
        })).ToArray();

        await Task.WhenAll(tasks);

        int sumAfter = _accountA.Read().Value + _accountB.Read().Value;
        AppendLog($"=== ИТОГ: A={_accountA.Read().Value}, B={_accountB.Read().Value}, сумма до={sumBefore},
после={sumAfter}, retry: {_totalConflictsDemo2} ===");
    }
}
```

7. Views/StmDrawable.cs

{} csharp

```csharp
using Microsoft.Maui.Graphics;
using StmVisualDemo.ViewModels;

namespace StmVisualDemo.Views;

public class StmDrawable : IDrawable
{
    private readonly MainViewModel _vm;
    public StmDrawable(MainViewModel vm) => _vm = vm;

    public void Draw(ICanvas canvas, RectF dirtyRect)
    {
        canvas.FillColor = Colors.White;
        canvas.FillRectangle(dirtyRect);

        if (_vm.IsSingleDemo)
            DrawSingleAccount(canvas, dirtyRect);
        else
            DrawTransfer(canvas, dirtyRect);
    }

    private void DrawSingleAccount(ICanvas canvas, RectF rect)
    {
        var center = new PointF(rect.Center.X, rect.Center.Y);
        const float boxSize = 110;

        var workers = _vm.Workers;
        int n = workers.Length;
        float radius = Math.Min(rect.Width, rect.Height) / 2f - 70;
        if (radius < 40) radius = 40;

        for (int i = 0; i < n; i++)
        {
            double angle = 2 * Math.PI * i / n - Math.PI / 2;
            float x = center.X + radius * (float)Math.Cos(angle);
            float y = center.Y + radius * (float)Math.Sin(angle);

            var color = ColorForState(workers[i].State);

            canvas.StrokeColor = color.WithAlpha(0.45f);
            canvas.StrokeSize = workers[i].State == WorkerState.Idle ? 1 : 2;
            canvas.DrawLine(x, y, center.X, center.Y);

            canvas.FillColor = color;
            canvas.FillCircle(x, y, 15);
            canvas.StrokeColor = Colors.Black;
            canvas.StrokeSize = 1;
            canvas.DrawCircle(x, y, 15);

            canvas.FontColor = Colors.Black;
            canvas.FontSize = 10;
            canvas.DrawString(workers[i].Id.ToString(), x - 15, y - 15, 30, 30,
                HorizontalAlignment.Center, VerticalAlignment.Center);
        }

        canvas.FillColor = Color.FromArgb("#1976D2");
        canvas.FillRoundedRectangle(center.X - boxSize / 2, center.Y - boxSize / 2, boxSize, boxSize, 16);
        canvas.StrokeColor = Colors.Black;
        canvas.StrokeSize = 1;
        canvas.DrawRoundedRectangle(center.X - boxSize / 2, center.Y - boxSize / 2, boxSize, boxSize, 16);

        canvas.FontColor = Colors.White;
        canvas.FontSize = 20;
        canvas.DrawString($"{_vm.SingleAccountBalance}", center.X - boxSize / 2, center.Y - boxSize / 2, boxSize,
boxSize,
            HorizontalAlignment.Center, VerticalAlignment.Center);

        DrawLegend(canvas, rect);
    }

    private void DrawTransfer(ICanvas canvas, RectF rect)
    {
        float boxW = 100, boxH = 90;
        var aCenter = new PointF(rect.Width * 0.25f, rect.Center.Y);
        var bCenter = new PointF(rect.Width * 0.75f, rect.Center.Y);

        canvas.FillColor = Color.FromArgb("#388E3C");
        canvas.FillRoundedRectangle(aCenter.X - boxW / 2, aCenter.Y - boxH / 2, boxW, boxH, 14);
        canvas.FontColor = Colors.White;
        canvas.FontSize = 16;
        canvas.DrawString($"A\n{_vm.AccountABalance}", aCenter.X - boxW / 2, aCenter.Y - boxH / 2, boxW, boxH,
            HorizontalAlignment.Center, VerticalAlignment.Center);

        canvas.FillColor = Color.FromArgb("#7B1FA2");
        canvas.FillRoundedRectangle(bCenter.X - boxW / 2, bCenter.Y - boxH / 2, boxW, boxH, 14);
        canvas.DrawString($"B\n{_vm.AccountBBalance}", bCenter.X - boxW / 2, bCenter.Y - boxH / 2, boxW, boxH,
            HorizontalAlignment.Center, VerticalAlignment.Center);

        var workers = _vm.Workers;
        int n = workers.Length;
        float spanX = bCenter.X - aCenter.X;

        for (int i = 0; i < n; i++)
        {
            float t = (i + 1f) / (n + 1f);
            float x = aCenter.X + spanX * t;
            bool aToB = workers[i].TargetAccount == 1;
            float y = rect.Center.Y + (aToB ? -1 : 1) * (40 + 20 * (i % 3));

            var color = ColorForState(workers[i].State);
            canvas.StrokeColor = color.WithAlpha(0.4f);
            canvas.StrokeSize = 2;

            var from = aToB ? aCenter : bCenter;
            var to = aToB ? bCenter : aCenter;
            canvas.DrawLine(x, y, from.X, from.Y);
            canvas.DrawLine(x, y, to.X, to.Y);

            canvas.FillColor = color;
            canvas.FillCircle(x, y, 12);
            canvas.StrokeColor = Colors.Black;
            canvas.StrokeSize = 1;
            canvas.DrawCircle(x, y, 12);
        }

        DrawLegend(canvas, rect);
    }

    private void DrawLegend(ICanvas canvas, RectF rect)
    {
        var items = new (WorkerState state, string text)[]
        {
            (WorkerState.Idle, "Ожидание"),
            (WorkerState.Reading, "Чтение"),
            (WorkerState.Computing, "Вычисление"),
            (WorkerState.Conflict, "Конфликт / Retry"),
            (WorkerState.Success, "Коммит")
        };

        float x = 10, y = rect.Height - 24 * items.Length - 10;
        canvas.FontSize = 11;
        foreach (var (state, text) in items)
        {
            canvas.FillColor = ColorForState(state);
            canvas.FillCircle(x + 6, y + 6, 6);
            canvas.FontColor = Colors.Black;
            canvas.DrawString(text, x + 18, y, 160, 18, HorizontalAlignment.Left, VerticalAlignment.Center);
            y += 20;
        }
    }

    private static Color ColorForState(WorkerState s) => s switch
    {
        WorkerState.Idle => Colors.Gray,
        WorkerState.Reading => Colors.DeepSkyBlue,
        WorkerState.Computing => Colors.Orange,
        WorkerState.Committing => Colors.Gold,
        WorkerState.Conflict => Colors.Red,
        WorkerState.Success => Colors.MediumSeaGreen,
        _ => Colors.Gray
    };
}
```

8. MainPage.xaml

{} XML

```xml
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:vm="clr-namespace:StmVisualDemo.ViewModels"
             x:Class="StmVisualDemo.MainPage"
             Title="STM / CAS Visual Demo">

    <ContentPage.BindingContext>
        <vm:MainViewModel />
    </ContentPage.BindingContext>

    <Grid RowDefinitions="Auto,Auto,*,Auto" Padding="12" RowSpacing="8">

        <VerticalStackLayout Grid.Row="0" Spacing="6">
            <Label Text="Лабораторная работа: Транзакционная память (STM) и CAS"
                   FontSize="16" FontAttributes="Bold" HorizontalTextAlignment="Center" />

            <Picker Title="Выберите демонстрацию" SelectedIndex="0"
SelectedIndexChanged="OnDemoPickerChanged">
                <Picker.Items>
                    <x:String>Демо 1: CAS (Interlocked.CompareExchange)</x:String>
                    <x:String>Демо 2: STM-перевод между счетами (MVCC)</x:String>
                </Picker.Items>
            </Picker>

            <Grid ColumnDefinitions="*,*" ColumnSpacing="8">
                <VerticalStackLayout Grid.Column="0">
                    <Label Text="{Binding WorkerCount, StringFormat='Потоков: {0:F0}'}" />
                    <Slider Minimum="2" Maximum="60" Value="{Binding WorkerCount, Mode=TwoWay}" />
                </VerticalStackLayout>
                <VerticalStackLayout Grid.Column="1" IsVisible="{Binding IsSingleDemo}">
                    <Label Text="{Binding DepositAmount, StringFormat='Сумма депозита: {0:F0}'}" />
                    <Slider Minimum="1" Maximum="100" Value="{Binding DepositAmount, Mode=TwoWay}" />
                </VerticalStackLayout>
                <VerticalStackLayout Grid.Column="1" IsVisible="{Binding IsTransferDemo}">
                    <Label Text="{Binding TransferAmount, StringFormat='Сумма перевода: {0:F0}'}" />
                    <Slider Minimum="1" Maximum="100" Value="{Binding TransferAmount, Mode=TwoWay}" />
                </VerticalStackLayout>
            </Grid>

            <HorizontalStackLayout Spacing="10" HorizontalOptions="Center">
                <Button Text="Запустить" Command="{Binding StartCommand}" IsEnabled="{Binding IsNotRunning}" />
                <Button Text="Сброс" Command="{Binding ResetCommand}" IsEnabled="{Binding IsNotRunning}" />
                <ActivityIndicator IsRunning="{Binding IsRunning}" IsVisible="{Binding IsRunning}" />
            </HorizontalStackLayout>
        </VerticalStackLayout>

        <Grid Grid.Row="1" ColumnDefinitions="*,*,*" ColumnSpacing="8">
            <Frame Grid.Column="0" Padding="8" BorderColor="LightGray" IsVisible="{Binding IsSingleDemo}">
                <Label Text="{Binding SingleAccountBalance, StringFormat='Баланс: {0}'}"
HorizontalTextAlignment="Center"/>
            </Frame>
            <Frame Grid.Column="0" Padding="8" BorderColor="LightGray" IsVisible="{Binding IsTransferDemo}">
                <Label Text="{Binding AccountABalance, StringFormat='Счёт A: {0}'}"
HorizontalTextAlignment="Center"/>
            </Frame>
            <Frame Grid.Column="1" Padding="8" BorderColor="LightGray" IsVisible="{Binding IsTransferDemo}">
                <Label Text="{Binding AccountBBalance, StringFormat='Счёт B: {0}'}"
HorizontalTextAlignment="Center"/>
            </Frame>
            <Frame Grid.Column="2" Padding="8" BorderColor="LightGray">
                <Label Text="{Binding ConflictsText}" HorizontalTextAlignment="Center" FontSize="12"/>
            </Frame>
        </Grid>

        <GraphicsView x:Name="CanvasView" Grid.Row="2" BackgroundColor="White" />

        <ScrollView Grid.Row="3" HeightRequest="140">
            <Label Text="{Binding LogText}" FontSize="11" FontFamily="Consolas" />
        </ScrollView>

    </Grid>
</ContentPage>
```

9. MainPage.xaml.cs

{} csharp

```csharp
using StmVisualDemo.ViewModels;
using StmVisualDemo.Views;

namespace StmVisualDemo;

public partial class MainPage : ContentPage
{
    private readonly MainViewModel _vm;
    private IDispatcherTimer? _timer;

    public MainPage()
    {
        InitializeComponent();
        _vm = (MainViewModel)BindingContext;
        CanvasView.Drawable = new StmDrawable(_vm);

        _timer = Dispatcher.CreateTimer();
        _timer.Interval = TimeSpan.FromMilliseconds(80);
        _timer.Tick += (s, e) =>
        {
            _vm.RefreshStats();
            CanvasView.Invalidate();
        };
        _timer.Start();
    }

    private void OnDemoPickerChanged(object sender, EventArgs e)
    {
        if (sender is Picker picker)
        {
            _vm.SelectedDemo = picker.SelectedIndex == 0
                ? DemoType.SingleAccountCas
                : DemoType.TransferStm;
        }
    }
}
```

Как это связано с теорией (для пояснительной записки)

Пункт теории	Где реализовано
Оптимистичный подход (vs pessimistic lock)	`CasAccount.Deposit` и `StmEngine.Transfer` — нет `lock` на время работы, только короткая блокировка на фазе commit
Чтение снимка / изоляция	`TVar<T>.Read()` возвращает снимок (Value, Version)
Валидация и коммит	`StmEngine.Transfer` : проверка версий перед записью
Откат и Retry при конфликте	`while(true)` + `onConflict` callback — возврат к чтению заново
MVCC (как в Clojure STM)	`TVar<T>` хранит версию вместе со значением
CAS (Compare-And-Swap)	`Interlocked.CompareExchange` в `CasAccount` — прямой аналог примера 1 из методички
Сохранение инварианта при параллелизме	Демо 2: сумма A+B остаётся постоянной (1500) при любом количестве параллельных потоков

Запуск

{} Bash

```bash
dotnet build -t:Run -f net8.0-android   # ИЛИ net8.0-windows10.0.19041.0, net8.0-maccatalyst
```

При нажатии **«Запустить»** в Демо 1 N потоков параллельно атакуют один `CasAccount` — видно, как часть кружков мигает красным (конфликт CAS) и повторяет попытку, итоговый баланс всегда корректен (`N × amount`). В Демо 2 потоки параллельно переводят деньги между A и B в обе стороны — сумма всегда сохраняется, конфликтные транзакции откатываются и перезапускаются.

Ask followup...

+ @ Chat

Inputs are processed by third-party AI and responses may be inaccurate.